OpenEarth-Agent: From Tool Calling to Tool Creation for Open-Environment Earth Observation

Sijie Zhao[1,2,†] **Feng Liu**[2,3,†] **Xueliang Zhang**[1*] **Hao Chen**[2*]
Xinyu Gu[2] Zhe Jiang[2] Fenghua Ling[2] Ben Fei[2] Wenlong Zhang[2]
Junjue Wang[4] Weihao Xuan[4] Pengfeng Xiao[1] Naoto Yokoya[4] Lei Bai[2]

[1]Nanjing University [2]Shanghai Artificial Intelligence Laboratory
[3]Shanghai Jiao Tong University [4]The University of Tokyo

† Equal contribution * Corresponding author

Abstract

Earth Observation (EO) is essential for perceiving dynamic land surface changes, yet deploying autonomous EO in open environments is severely hindered by the immense diversity of multi-source data and heterogeneous tasks. While remote sensing agents have emerged to streamline EO workflows, existing tool-calling agents are confined to closed environments. They rely on pre-defined tools and are restricted to narrow scope, limiting their generalization to the diverse data and tasks. To overcome these limitations, we introduce **OpenEarth-Agent**, the first tool-creation agent framework tailored for open-environment EO. Rather than calling predefined tools, OpenEarth-Agent employs adaptive workflow planning and tool creation to generate specialized tools tailored to unseen data and tasks. This adaptability is bolstered by an open-ended integration of multi-stage tools and cross-domain knowledge bases, enabling robust execution in the entire EO pipeline across multiple application domains. To comprehensively evaluate EO agents in open environments, we propose **OpenEarth-Bench**, a novel benchmark comprising 596 real-world, full-pipeline cases across seven application domains, explicitly designed to assess agents' adaptive planning and tool creation capabilities. Only essential pre-trained model tools are provided in this benchmark, devoid of any other predefined task-specific tools. Extensive experiments demonstrate that OpenEarth-Agent successfully masters full-pipeline EO across multiple domains in the open environment. Notably, on the cross-benchmark Earth-Bench Feng et al. (2025), our tool-creating agent equipped with 6 essential pre-trained models achieves performance comparable to tool-calling agents relying on 104 specialized tools, and significantly outperforms them when provided with the complete toolset. In several cases, the created tools exhibit superior robustness to data anomalies compared to human-engineered counterparts, highlighting the potential of tool-creating agents for advanced EO. Code and Benchmark wiil be available at OpenEarth-Agent.

1 Introduction

Earth Observation (EO) serves as a core paradigm for perceiving dynamic land surface changes and understanding human-earth systems, aiming to transform massive raw observation signals into actionable insights within open environments Yuan et al. (2020). The intrinsic nature of the open environment is fundamentally characterized by the diversity of EO data and tasks Xiong et al. (2024b).

Preprint. Under review.

Specifically, the data encompasses multi-modal observations (e.g., multispectral, Synthetic Aperture Radar (SAR)) and diverse derived products (e.g., land cover, wildfire) Xiong et al. (2024a, 2025). Concurrently, EO tasks entail the diverse analysis of heterogeneous land surface elements, including urban, forests, and soils Brown et al. (2022). In such an open environment, EO necessitates a long-horizon pipeline ranging from data preparation to geospatial analysis, and finds extensive applications across diverse domains, such as urban studies and agriculture Gorelick et al. (2017). Consequently, this poses substantial challenges for autonomous EO in open environments.

The escalating complexity of EO has catalyzed the development of tool-calling agents within the remote sensing domain Xu et al. (2024); Guo et al. (2024); Hu et al. (2025). By planning and calling predefined external tools to execute workflows, remote sensing agents decompose complex EO tasks into node-centric executions along a fixed toolchain, thereby facilitating autonomous and efficient EO Shabbir et al. (2025); Feng et al. (2025); Li et al. (2025); Chen et al. (2025). However, existing remote sensing agents predominantly operate within restricted, closed environments, focusing on the planning and invocation of predefined tools, as shown in Fig. 1. When deployed in expansive open environments, they exhibit two fundamental limitations: **1) Closed Nature of Tool Calling Limits Data and Task Generalization:** EO in open environments involves a high degree of data and task diversity, making it intractable to construct predefined tools for all potential scenarios. Nevertheless, current agents are restricted to calling a given set of tools and rely on these fixed tools as nodes for workflow planning. When a data or task in the open environment demands previously unseen functionality, the agent is unable to execute. **2) Restricted Operational Scopes Limit End-to-End Cross-Domain Generalization:** Open-environment EO encompasses a long pipeline and spans multiple cross domains. However, existing agents are typically restricted to a few specific stages and domains. Rather than autonomously acquiring and preprocessing data, they often read prepared datasets directly, executing only feature extraction or rudimentary analytical tasks. Furthermore, their cross-domain capabilities are limited to a narrow set of domain-specific cases, lacking the generalizability for EO across multiple domains.

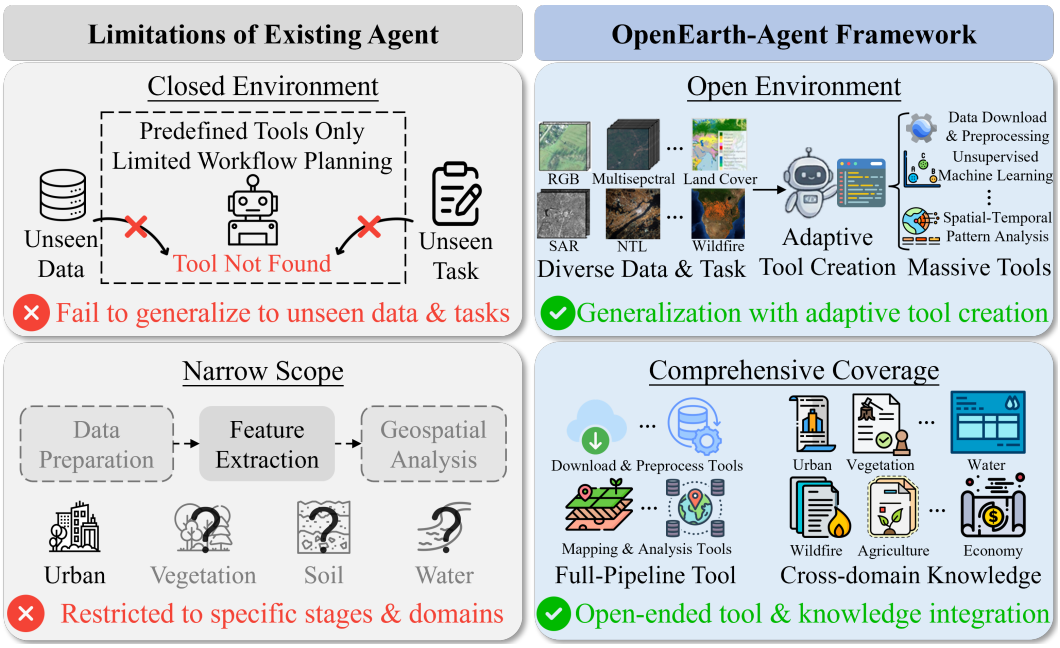

Figure 1: Overview of existing limitations and proposed solutions. Existing EO agents are failing to generalize to unseen data and tasks due to calling predefined tools, and are restricted to specific EO stages and application domains. In contrast, the proposed OpenEarth-Agent generalize to diverse data and tasks by creating adaptive tools, achieving full EO workflow and cross-domain coverage through open-ended tool and knowledge integration.

To address these challenges, we propose **OpenEarth-Agent**, the first tool-creation agent framework tailored for open-environment EO, as shown in Fig. 1. OpenEarth-Agent tackles the aforementioned limitations through two core mechanisms: **(1) Adaptive Workflow Planning and Tool Creation:**

To overcome the generalization bottleneck inherent in predefined tool calling, OpenEarth-Agent replaces static tool calling with a dynamic tool creation paradigm. Unlike conventional approaches that rely on simple code generation for tool creation, OpenEarth-Agent performs real-time data exploration and task logic planning to provide comprehensive and structured data and task contexts. Furthermore, upon tool creation and task execution, it leverages domain-specific geoscience knowledge for result verification and iteratively optimizes the synthesized tools. Specifically, confronted with diverse data and tasks, OpenEarth-Agent first synthesizes code-probing tools to perceive the metadata and spatio-temporal-modal characteristics of multi-source data in real time. This crucial grounding step ensures that the subsequently created tools are strictly calibrated to correctly process heterogeneous data. Conditioned on these data insights and task context, the agent then performs adaptive workflow planning, structuring the task execution into a Directed Acyclic Graph (DAG). Rather than calling fixed tools, OpenEarth-Agent dynamically creates adaptive tools explicitly tailored to the specific data and task requirements of each DAG node. Finally, through an iterative feedback mechanism, OpenEarth-Agent continuously refines these generated tools, fundamentally empowering it to seamlessly generalize to unseen data and novel tasks in open environments. **(2) Open-Ended Tool and Knowledge Integration:** To bolster the agent's capabilities across the entire EO pipeline and various application domains, OpenEarth-Agent incorporates extensive tool and knowledge integration. Departing from a rigid adherence to predefined workflows and static tool calling, OpenEarth-Agent leverages these resources as critical references to enable dynamic planning and real-time tool synthesis, adapting to heterogeneous data and complex tasks. Regarding tools, it integrates specialized functions and templates for multi-source data acquisition and preprocessing (e.g. multispectral, SAR, nighttime light), multi-paradigm feature extraction (e.g. index calculation, unsupervised clustering, pre-trained model calling), and multi-dimensional geospatial analysis (e.g. spatial pattern, temporal trend), comprising $66,736$ functional tools and 514 execution templates. For knowledge integration, it encompasses $11,694$ static knowledge bases across domains like urban, agriculture, and soils, while concurrently supporting real-time online knowledge retrieval.

To comprehensively evaluate the performance of remote sensing agents in open environments, we construct **OpenEarth-Bench**, the first open-environment evaluation benchmark for EO, as shown in Fig. 3. Previous benchmarks typically conceptualize predefined tools as nodes, linking them to form cases within closed environments and assess the agent's tool planning and invocation capabilities. In contrast, OpenEarth-Bench evaluates the intrinsic ability to adaptively plan workflow and create tools without relying on predefined tools in open environments. OpenEarth-Bench comprises 596 real-world application cases across 7 application domains (e.g. urban, agriculture, vegetation). Each case encapsulates a full EO pipeline from data preparation and feature extraction to geospatial analysis. To ensure comprehensive coverage of diverse EO data and tasks, the benchmark includes the acquisition and preprocessing of multi-source observations and products; feature extraction utilizing statistical learning, machine learning, and deep learning methodologies; and in-depth geospatial analysis across multiple dimensions, such as temporal trend analysis, spatial pattern analysis, and spatio-temporal coupling analysis.

To assess the efficacy of OpenEarth-Agent in open environments, we evaluate its performance powered by various Large Language Models (LLMs) on OpenEarth-Bench. Furthermore, to rigorously evaluate its workflow planning and tool creation capabilities, we conduct cross-benchmark experiments on Earth-Bench. To isolate the effects of knowledge and tool integration, we configure OpenEarth-Agent without any external knowledge or extraneous tools in this benchmark. Under this controlled setting, we compare OpenEarth-Agent equipped with either only 6 essential pre-trained model tools or all 104 tools from the benchmark, against other agents that integrate and invoke the complete toolset.

Overall, the primary contributions of this work are summarized as follows:

- We propose OpenEarth-Agent, the first remote sensing agent architecture designed for open environments. By performing adaptive workflow planning and tool creation conditioned on data and task contexts, OpenEarth-Agent accommodates diverse EO data and tasks. Through multi-stage tool and cross-domain knowledge integration, it effectively executes full-pipeline EO across multiple application domains.

- We construct OpenEarth-Bench, the first remote sensing agent benchmark oriented towards open environments. Comprising 596 full-pipeline cases sourced from real-world applications across various domains, it serves as a robust platform for evaluating agent performance in open environments.

- Extensive experiments on OpenEarth-Bench and cross-benchmark evaluations on Earth-Bench validate the effectiveness of OpenEarth-Agent. Notably, on Earth-Bench, OpenEarth-Agent demonstrates the ability to create functionally equivalent professional tools, and in certain instances, tools with superior data adaptability. Operating with merely 6 integrated tools, it achieves performance comparable to agents utilizing full tool calling, and significantly outperforms existing agents when provided with the complete toolset.

2 Related Works

2.1 Remote Sensing Agents for Earth Observation

Early applications of LLMs in EO predominantly relied on developing domain-specific remote sensing encoders for end-to-end multimodal alignment Liu et al. (2024b); Zhang et al. (2024). However, when confronted with heterogeneous remote sensing data and tasks, this paradigm is highly expensive to scale and relies heavily on exhaustive fine-tuning Do et al. (2025). Recent research has progressively shifted towards constructing tool-augmented agents. By calling and orchestrating predefined external tools, these agents execute designated workflows, thereby transforming complex EO tasks into node-based executions within a fixed toolchain Shabbir et al. (2025); Chen et al. (2025); Feng et al. (2025); Li et al. (2025); Dai et al. (2026). Nevertheless, most existing remote sensing agents are confined to idealized, closed-world testing environments. They depend on static, predefined toolsets for restricted planning and are typically constrained to isolated procedural stages within limited domains. Consequently, they struggle to address the cross-domain and full-pipeline demands of real-world EO. Thus, there is a pressing need to develop open-environment EO agents capable of autonomous operation across diverse domains and entire workflows.

2.2 Remote Sensing Agent Benchmarks

Early remote sensing multimodal benchmarks integrating LLMs primarily evaluated the models' fundamental perception and reasoning capabilities An et al. (2024); Danish et al. (2025); Wang et al. (2025). Recent efforts have begun to construct benchmarks specifically for remote sensing agents, shifting the focus towards evaluating tool planning and invocation capabilities following the integration of specialized tools Shabbir et al. (2025); Chen et al. (2025); Feng et al. (2025); Li et al. (2025). However, these existing benchmarks are generally designed around closed environments, requiring agents to orchestrate workflows strictly within a predefined set of tools. Consequently, such benchmarks remain restricted to a limited number of pre-configured scenarios, lacking a comprehensive evaluation of full EO pipelines and cross-domain applications in open environments. There is currently an urgent need to establish benchmarks that rigorously assess the real-world capabilities of EO agents in open environments.

3 Methodology

3.1 Multi-Agent Architecture

To address the high diversity and complexity of EO data and tasks in open environments, we propose OpenEarth-Agent, a multi-agent collaborative framework, as shown in Fig. 2. Unlike existing tool-calling agents that rely on predefined toolsets and are restricted to narrow scope, our architecture aims to achieve an autonomous closed-loop for the entire EO pipeline across multiple domains through adaptive workflow planning and tool creation. OpenEarth-Agent operates collaboratively via five core agents: Data Summary Agent, Planning Agent, Workflow Agent, Coding Agent, and Checking Agent. The overall operational logic can be divided into the following three core stages:

1) Iterative Data Exploration and Contextualization: In open environments, flexibly adapting to multi-source remote sensing data is a prerequisite for autonomous EO. Given the natural language task instruction $\mathcal{Q}$, the Data Summary Agent first retrieves and acquires the relevant raw data. Rather than relying on static parsers, the agent dynamically generates and executes specialized data-probing tools to extract a comprehensive data profile $\mathcal{D}$. This profiling captures both essential metadata (e.g., file paths, dimensions, spectral bands, and spatial projections) and complex spatial-temporal-modal features. Crucially, this exploration process is strictly iterative: if a probing tool encounters an

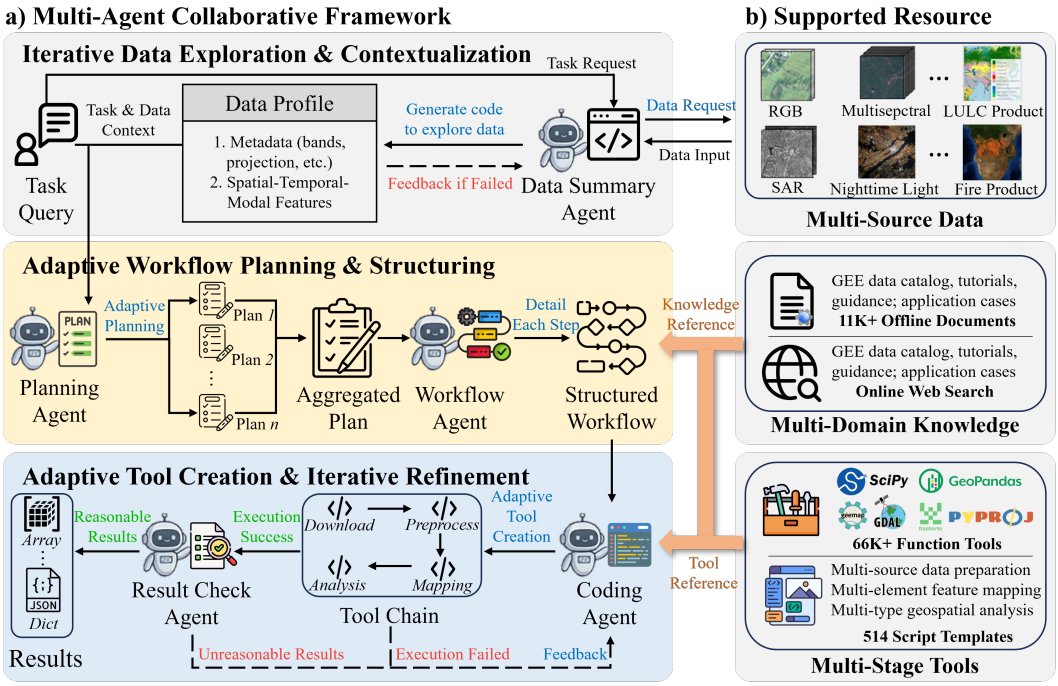

Figure 2: Overview of the OpenEarth-Agent. The OpenEarth-Agent performs real-time perception of multi-source EO data, followed by adaptive workflow planning and structuring based on the task and data context. It then proceeds with adaptive tool creation and execution, incorporating real-time inspection and feedback on the execution process and outcomes to generate reasonable results.

execution error due to underlying data irregularities, the agent autonomously analyzes the traceback, refines the script, and re-executes the tool until the data is successfully parsed. Finally, the agent distills the extracted characteristics into a rich, structured data context, establishing an accurate and reliable informational foundation for subsequent workflow planning and tool creation.

2) Adaptive Workflow Planning and Structuring: After data contextualization, the Planning Agent receives the natural language task instruction $\mathcal{Q}$, and the aforementioned data profile $\mathcal{D}$, leveraging the integrated cross-domain expert knowledge base $\mathcal{K}$ and multi-stage tools $\mathcal{T}$ to generate a reasonable plan. This agent is responsible for decoupling the macroscopic task into a sequence of subtasks with a clear hierarchical structure and execution order. To effectively mitigate the planning hallucination problem common in LLMs, we introduce a multiple-plan aggregation mechanism. The Planning Agent first generates a set of n candidate execution blueprints, denoted as $\mathcal{P} = \{p_1, p_2, \ldots, p_n\}$. It then merges execution paths with similar semantics and prioritizes them based on dimensions such as data availability and logical rigor, ultimately outputting the aggregated optimal task plan p^*. This process can be formalized as:

$$p^* = \mathcal{A}(\mathcal{P}), \quad \mathcal{P} = \mathcal{G}(\mathcal{Q}, \mathcal{D}, \mathcal{K}, \mathcal{T}) \tag{1}$$

where $\mathcal{G}$ denotes the candidate plan generation function based on the task and data context, and $\mathcal{A}$ represents the plan aggregation and ranking function.

To bridge the semantic gap between macroscopic task planning and microscopic code generation, the Workflow Agent transforms the optimal plan p^* into a structured computational workflow. Specifically, this agent decomposes the overall plan into multiple independent execution script nodes with well-defined functions. It strictly regulates the data flow dependencies between nodes and explicitly defines the input/output interface information and key execution parameters for each script node. This process instantiates the natural language planning blueprint into a logically rigorous DAG, providing a precise standard reference for subsequent code generation.

3) Adaptive Tool Creation and Iterative Refinement: Guided by the structured workflow, the system proceeds to the tool creation and execution phase. Following the topological dependency order of the workflow, the Coding Agent sequentially generates execution scripts for each node. Unlike existing

methods that retrieve and invoke tools from a limited toolset, the Coding Agent fully leverages the code generation capabilities of LLMs, combined with the integrated professional toolsets and knowledge base, to create adaptive and specialized tools within open environments. The agent advances to the tool creation for the next node only after the script of the preceding node has been successfully executed.

To ensure the robustness of the system in open environments, the Checking Agent constructs a feedback-driven validation closed loop. As the Coding Agent executes each script, the Checking Agent performs runtime diagnostics to determine whether the code runs successfully and utilizes geoscientific rules to verify the validity of the results (e.g., checking if saved results consist entirely of invalid values or if the calculated remote sensing indices fall outside logical numerical ranges). Once an anomaly is detected, the diagnostic feedback is transmitted back to the Coding Agent, triggering parameter correction and code refactoring mechanisms to initiate a new round of tool creation and execution. Letting the code output at the k-th round be $O^{(k)}$, this autonomous error-correction closed loop can be modeled as:

$$O^{(k+1)} = \mathcal{E}\left(\mathcal{M}\left(\mathcal{C}^{(k)}, \mathcal{F}^{(k)}\right), \mathcal{D}\right), \quad \text{s.t.} \quad \mathcal{V}(O^{(k+1)}) = 1 \tag{2}$$

where $\mathcal{C}^{(k)}$ is the code generated in the k-th round, $\mathcal{F}^{(k)}$ is the anomaly feedback information provided by the Checking Agent, $\mathcal{M}$ denotes the code modifier, $\mathcal{E}$ represents the code executor, and $\mathcal{V}$ is the validation function used to determine whether the execution result meets the convergence criteria. This iterative feedback mechanism significantly enhances the robustness and scientific rigor of OpenEarth-Agent in handling complex tasks.

3.2 Full-Pipeline Tool Integration

To ensure that OpenEarth-Agent can reasonably create tools throughout the entire EO pipeline in open environments, we design a highly flexible tool integration paradigm. We construct the tool integration as a two-tier architecture comprising standardized software packages and an expert-level code script library, serving as the reference for tool creation.

At the standardized software package level, we integrate 66,736 function tools from 11 Python libraries across the domains of remote sensing, geocomputation, and general data science. This allows the agents to directly import and freely combine them within a standard Python environment to handle diverse foundational tasks. At the expert-level code script library level, we construct a modular repository validated by domain experts, covering complete or partial workflows for remote sensing applications, comprising 514 reference scripts. To enable autonomous invocation by the agents, we introduce a retrieval mechanism based on semantic similarity. First, an LLM generates a functional description for each script, which is then transformed into high-dimensional vectors via an embedding model to build a retrieval database. When an agent faces a new task, the system translates its requirements into natural language descriptions and performs vectorized matching. This retrieves the most relevant script pairs as high-quality reference examples and code templates, thereby significantly enhancing the accuracy and robustness of the agent's tool creation in remote sensing tasks.

Based on the aforementioned two-tier architecture, we conduct deep integration of specialized tools across three core stages of EO:

1) Multi-Source Data Acquisition and Preprocessing: Facing complex data sources in open environments, OpenEarth-Agent integrates a comprehensive toolchain covering cloud-based acquisition and local preprocessing. For cloud-based data acquisition, it integrates interaction codes and application paradigms from the Google Earth Engine (GEE) platform. This enables the agent to dynamically acquire multi-modal raw observation data (e.g. multispectral data, nighttime light data and supports the direct invocation of global high-level remote sensing products (such as, land cover products, and wildfire products). For local data preprocessing, it comprehensively integrates function tools and operational templates from core spatial processing libraries like `gdal`, `rasterio`, `geopandas`, and `pyproj`, providing standardized code references for preprocessing operations such as resampling, projection transformation of multi-source data.

2) Multi-Paradigm Feature Extraction: To support the efficient transformation from massive pixels to element features, OpenEarth-Agent integrates multi-level feature extraction tools ranging from statistical learning, machine learning to deep learning. At the statistical and machine learn-

ing level, it integrates powerful general-purpose algorithm libraries such as `scikit-learn` and `scikit-image`, enabling the agent to rapidly construct feature extraction models. At the deep learning level, it incorporates pre-trained model tools for tasks like building semantic segmentation and multi-temporal change detection.

3) Multi-Dimensional Geospatial Analysis: To transform feature extraction results into a deep cognitive understanding of element states, OpenEarth-Agent integrates rich geostatistical analysis tools centered around three dimensions (temporal trends, spatial patterns, and spatiotemporal coupling). For time series analysis, it integrates libraries like `statsmodels` and `SciPy` to provide advanced statistical tool templates, such as autoregressive analysis and time series decomposition, to mine the long-term evolutionary patterns of element features. For spatial pattern and spatiotemporal coupling analysis, it deeply integrates the PySAL ecosystem, offering reference scripts for analysis like spatial autocorrelation analysis, hotspot analysis.

3.3 Multi-Domain Knowledge Integration

To address the complex and diverse EO demands in open environments, OpenEarth-Agent integrates knowledge from multiple remote sensing application domains, aiming to provide expert-level prior knowledge guidance for workflow planning and tool creation. Specifically, the integrated knowledge encompasses the GEE Data Catalog, tutorial documents, community documentation, and domain-specific knowledge extensively covering various remote sensing intersections, totaling 11,694 documents. Regarding the integration mechanism, we adopt a dual-branch knowledge retrieval architecture combining offline and online strategies:

1) Offline Knowledge Base: For multi-source heterogeneous GEE documentation and various remote sensing application cases, we segment massive long texts into independent semantic chunks and vectorize each chunk to construct a dedicated document vector database. This database provides a natural language query interface. During the workflow planning and tool creation, they can initiate queries in natural language, and the system will precisely recall the highly relevant textual chunks.

2) Online Retrieval Tool: Given the rapid iterations of remote sensing data and analytical methods in the real world, we equip the knowledge base with an online web search tool. When agents encounter knowledge blind spots, they can autonomously invoke this tool for real-time web retrieval. This allows OpenEarth-Agent to continuously acquire the latest academic literature, open-source code repositories, and updated API states at any time, ensuring the real-time nature of the system's knowledge and the continuous expansion of its boundaries.

4 OpenEarth-Bench

4.1 Overview of OpenEarth-Bench

To comprehensively evaluate the autonomous EO capabilities of agents in open environments, we construct OpenEarth-Bench, the first open-domain evaluation benchmark for remote sensing agents. Unlike existing benchmarks that test the tool planning and invocation capabilities of remote sensing agents in closed environments, OpenEarth-Bench is designed to evaluate the adaptive workflow planning and tool creation capabilities of agents in open environments across full EO pipelines and diverse application domains, as shown in Fig. 3.

To systematically examine the comprehensive performance of agents, OpenEarth-Bench comprises 596 application cases derived from real-world research, spanning seven remote sensing application domains: urban, agriculture, vegetation, water bodies, soil, economy, and snow. Each case covers a complete EO pipeline. The statistics is illustrated in Bottom left of Fig. 3. In the data preparation stage, the benchmark encompasses the automatic acquisition and preprocessing of multi-source data, including RGB, multi-spectral, SAR, NTL, and remote sensing products. In the feature extraction stage, it requires agents to flexibly employ diverse methods—ranging from statistical learning and machine learning to deep learning—to accomplish the feature extraction. In the geospatial analysis stage, the benchmark involves multi-dimensional scientific computations, such as temporal trend analysis, spatial correlation analysis, and spatio-temporal coupling analysis.

To systematically evaluate the performance of agents in open-environment EO, OpenEarth-Bench is designed with three distinctive characteristics: **1) Extensive Domain Coverage**: OpenEarth-Bench

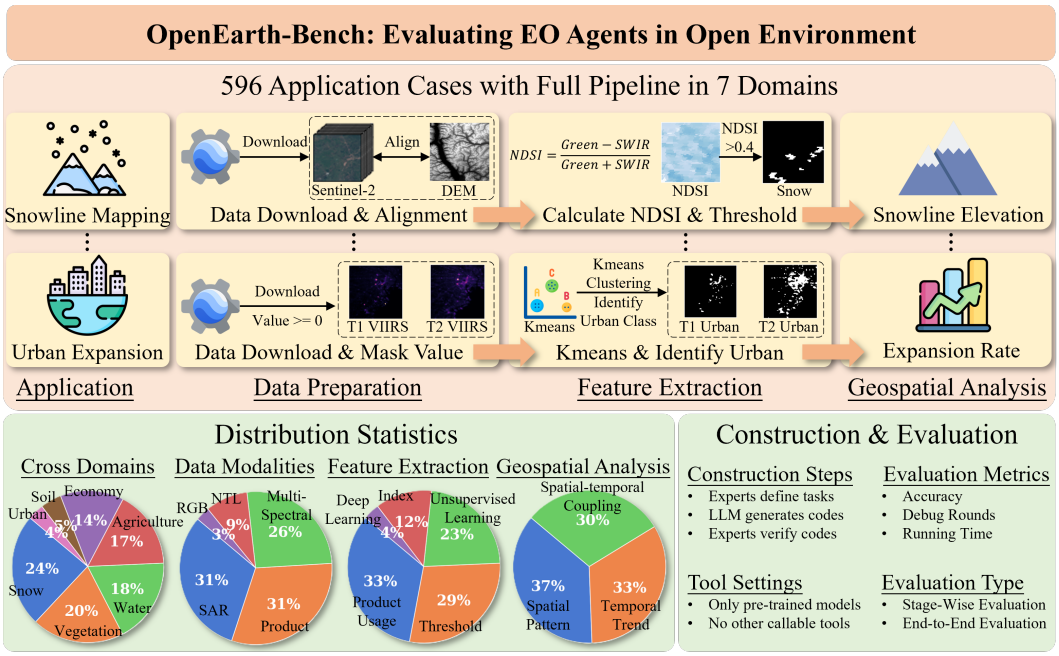

Figure 3: Overview of the OpenEarth-Bench. This benchmark comprises 596 real-world application cases spanning seven domains. Each case encompasses the entire workflow, ranging from data preparation and feature extraction to geospatial analysis.

comprises 596 application cases derived from real-world research, spanning seven remote sensing domains: urban, agriculture, vegetation, water bodies, soil, economy, and snow. Detailed distribution statistics are illustrated in the bottom left of Fig. 3. **2) Comprehensive Pipeline Coverage:** Each case in OpenEarth-Bench encompasses a complete EO pipeline with full coverage across all stages. In the data preparation stage, the benchmark includes the autonomous acquisition and preprocessing of multi-source data, such as RGB, multi-spectral, SAR, NTL, and other remote sensing products. In the feature extraction stage, agents are required to flexibly employ diverse methodologies—ranging from statistical and machine learning to deep learning. Finally, the geospatial analysis stage involves multi-dimensional scientific computations, including temporal trend analysis, spatial correlation, and spatio-temporal coupling analysis. **3) Minimalist Tool Provision:** OpenEarth-Bench provides only large-scale remote sensing models as essential tools for specific functionalities. No auxiliary tools are provided, requiring agents to autonomously synthesize necessary tools based on the data and task context in open environments.

Table 1: Comparison between OpenEarth-Bench and existing remote sensing agent benchmarks.

Benchmark	Data Preparation	Feature Extraction	Geospatial Analysis	Open Environment	Data Modalities
ThinkGeo Shabbir et al. (2025)	✗	✓	✗	✗	RGB
Earth-Bench Feng et al. (2025)	✗	✓	✓	✗	RGB, Multi-Spectral, Product
GeoPlan-Bench Li et al. (2025)	✓	✓	✓	✗	-
Cangling-Know. Chen et al. (2025)	✗	✓	✓	✗	RGB, Multi-Spectral, SAR
OpenEarth-Bench (Ours)	✓	✓	✓	✓	RGB, Multi-Spectral, SAR, NTL, Product

Table 1 details the comparison between OpenEarth-Bench and existing remote sensing agent benchmarks. As shown in the table, existing works are restricted by closed tool calling environments and are limited to a narrow scope of EO stages and data modalities. In contrast, OpenEarth-Bench achieves multi-source data support, full-pipeline coverage, and real-case evaluation in open environments for the first time. This provides a comprehensive and highly challenging testing platform to facilitate the development of remote sensing agents.

4.2 Benchmark Construction and Evaluation

To ensure the authenticity and high quality of OpenEarth-Bench in open environments, we adopt a collaborative construction paradigm consisting of "expert guidance, code generation, and human verification". First, senior domain experts led the definition and workflow of representative EO task cases from practical research across seven remote sensing application domains. Subsequently, leveraging advanced LLMs as an auxiliary engine, we preliminarily generated execution codes and workflow logic covering the entire "data preparation - feature extraction - geospatial analysis" pipeline based on the expert-defined task contexts. Finally, domain experts conducted thorough manual reviews and corrections of the model-generated codes, task performing logic, and final execution results. This process significantly improves the efficiency of benchmark construction while ensuring the scientific rigor and executability of all cases in open environments.

To facilitate evaluation in realistic open environments, *OpenEarth-Bench* exclusively provides pre-trained models tailored to specific cases, omitting other predefined callable tools. This design compels the agent to adaptively plan workflows and generate tools, conditioned on the data and task context. Given the inherently long-horizon nature of EO workflows, we design our evaluation protocol to assess the agent both at individual stages and across the entire pipeline: **1) Stage-wise Evaluation:** We decouple the comprehensive EO task into three independent stages: data preparation, feature extraction, and geospatial analysis, assessing the agent's performance independently within each phase. **2) End-to-end Evaluation:** We evaluate the agent's performance across all phases as it autonomously executes the complete EO pipeline. In this setting, errors in preceding stages naturally cascade and impact subsequent operations.

To multidimensionally quantify the comprehensive efficacy of *OpenEarth-Agent* in open environments, we construct an evaluation metric system encompassing effectiveness, reflective capability, and operational cost: **1) Accuracy:** Serves as the primary metric for task completion, evaluating whether the agent's output is successfully generated, correctly localized in designated storage, and strictly aligned with the ground truth. In the data preparation, we verify the consistency of metadata (e.g., data source, spatio-temporal settings) and ensure numerical discrepancies remain within a predefined error margin. In the feature extraction phase, we validate metadata alignment (e.g., value ranges, channel configurations) alongside numerical fidelity. For geospatial analysis, we confirm that the extracted insights are correctly structured as key-value pairs, with numerical deviations constrained within the acceptable tolerance. **2) Debug Rounds:** Records the number of iterations required for the agent to self-adjust and correct code based on environmental feedback when execution errors occur. This metric directly reflects the agent's capacity for self-reflection and error recovery. **3) Running Time:** Measures the total duration of the agent's planning-execution-feedback loop, evaluating the agent's operational efficiency in real-world scenarios. As cascading errors in the end-to-end evaluation can lead to disproportionate increases in debug rounds and running time for subsequent stages, we exclusively report these two metrics in the stage-wise evaluation. Accuracy is applied to both the stage-wise and end-to-end evaluations.

5 Experiment

5.1 Evaluations with different LLMs on OpenEarth-Bench

To evaluate the agent's performance in open environments, we benchmark OpenEarth-Agent on OpenEarth-Bench, supported by six advanced LLMs. These include two closed-source LLMs (GPT-5, Gemini-2.5-Flash Comanici et al. (2025)) and four open-source LLMs (Kimik2 Team et al. (2025), DeepSeek-V3.1 Liu et al. (2024a), Qwen3-Max Yang et al. (2025), Seed-1.6). Through an in-depth comparison of stage-wise and end-to-end evaluation results, as presented in Table 2 and Fig. 4, we draw the following observations:

1) Effectiveness in open environments: Despite the diverse data and tasks in open environments, and without relying on predefined tool APIs, Table 2 demonstrates that OpenEarth-Agent achieves high accuracy across all stages in the stage-wise evaluation, with GPT-5 yielding the best performance. Furthermore, the agent leverages result-checking feedback to steadily enhance its capability. Fig. 4 illustrates the self-correction proficiency of the evaluated LLMs, highlighting GPT-5's superior reflection and debugging abilities, which require fewer than five debug rounds across all stages.

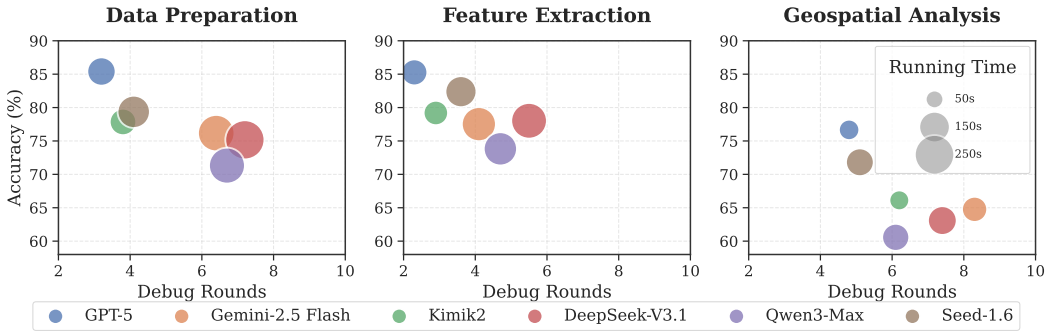

Figure 4: Results of OpenEarth-Agent based on different LLMs in Stage-Wise evaluation. The closer to the upper-left corner, the better.

Table 2: Accuracy comparison of OpenEarth-Agent powered by different LLMs on OpenEarth-Bench.

LLM	Stage-Wise			End-to-End		
	Data Preparation	Feature Extraction	Geospatial Analysis	Data Preparation	Feature Extraction	Geospatial Analysis
GPT-5	**85.40**	**85.27**	**76.66**	**82.38**	**74.83**	**58.72**
Gemini-2.5-Flash	76.17	77.52	64.76	74.16	61.58	45.47
Kimik2	77.85	79.19	66.11	75.84	64.09	47.81
DeepSeek-V3.1	75.17	78.02	63.08	72.82	60.74	43.12
Qwen3-Max	71.31	73.82	60.57	69.79	56.38	39.26
Seed-1.6	79.36	82.38	71.81	77.18	68.12	51.84

2) Complexity of cross-domain scientific analysis: In the stage-wise evaluation, compared to the high accuracies in Data Preparation and Feature Extraction, all models exhibit a noticeable performance degradation in the Geospatial Analysis stage. The best-performing model, GPT-5, achieves an accuracy of 76.66%, while the open-source LLMs generally hover around 60%–70%. This disparity reflects the fundamental difference in requirements between deep geospatial analysis and shallow data processing. While data preparation and feature extraction predominantly rely on general programming logic and mature algorithm invocations, geospatial analysis necessitates cross-domain expertise and profound analytical comprehension.

3) Impact of cascading errors in the full pipeline: In the end-to-end evaluation, all LLMs demonstrate a baseline capability for full-pipeline execution, validating OpenEarth-Agent's potential for comprehensive cross-stage EO. However, this evaluation also reveals the issue of cascading errors inherent in long-horizon tasks. Since errors in preceding stages are progressively amplified downstream, the accuracy in the feature extraction and geospatial analysis stages drops significantly compared to the stage-wise evaluation. For instance, GPT-5's geospatial analysis accuracy sharply decreases from 76.66% to 58.72%, underscoring the substantial challenges agents face in end-to-end EO.

4) Execution time in real-world environments: The runtime statistics presented in Fig. 4 realistically reflect the agent's efficiency in open environments. All LLMs consume considerable time during the data preparation and feature extraction stages. This is primarily because data preparation involves the online retrieval and downloading of multi-source remote sensing imagery, whereas feature extraction entails online unsupervised learning and the inference of pre-trained models. Conversely, the time required for the geospatial analysis stage is significantly reduced, as it predominantly involves the analysis of structured feature data.

5.2 Cross-benchmark evaluations on Earth-Bench

To further validate the capabilities of OpenEarth-Agent in open environments, we conduct experiments on the cross-domain benchmark Earth-Bench Feng et al. (2025) and compare it with the tool-calling agent, Earth-Agent Feng et al. (2025), across various LLMs. Earth-Bench comprises 104 professional

tools, including pre-trained models and programmatic function tools. To ensure a fair comparison, we disabled all external knowledge and tool integration within OpenEarth-Agent. We evaluate OpenEarth-Agent under two distinct settings: 1) integrating only six essential pre-trained models from Earth-Bench to evaluate its capacity to adaptively create functionally equivalent tools, plan, and execute tasks in open environments devoid of predefined tools; and 2) integrating all available tools in Earth-Bench to assess its tool planning and invocation capabilities in a resource-rich environment. Since OpenEarth-Agent does not involve tool calls, we only use the Accuracy of this benchmark as a validation metric.

The quantitative results, presented in Table 3, demonstrate that even when equipped with a minimal set of pre-trained models, OpenEarth-Agent achieves performance comparable to Earth-Agent across all evaluated LLMs. This indicates that in open environments lacking accessible tools, OpenEarth-Agent can effectively process diverse data and tasks by adaptively generating functionally equivalent tools, matching the efficacy of agents reliant on predefined tools. Furthermore, when equipped with the complete toolset, OpenEarth-Agent significantly outperforms Earth-Agent across all LLMs, highlighting its superiority over existing tool-calling agents in tool-abundant environments.

Table 3: Accuracy comparison on Earth-Bench across different LLM backbones.

LLM	Earth-Agent	OpenEarth-Agent	
		6 Tools	Full Tools
GPT-5	63.16	59.92	**67.61**
Gemini-2.5-Flash	55.06	53.44	**57.89**
Kimik2	56.68	52.63	**59.11**
DeepSeek-V3.1	52.23	51.02	**56.68**
Qwen3-Max	47.37	46.97	**52.63**
Seed-1.6	59.51	52.63	**61.94**

Beyond functional consistency, the tools autonomously created by OpenEarth-Agent exhibit significantly superior data adaptability compared to predefined tools. Specifically, the predefined tools in Earth-Bench are strictly tailored to the benchmark's specific data distributions. When applied to diverse data in open environments under identical task settings, they frequently encounter three critical limitations: **1) Hard-coded sensor parameters:** Numerous tools rigidify parameters for specific sensors, rendering them incompatible with alternative data sources; for instance, `compute_tvdi` and `calculate_ndsi` hard-code MODIS-specific scaling factors, while `lst_single_channel` and `temperature_emissivity_separation` fix specific band wavelengths. **2) Omission of invalid value masking:** Tools such as `calculate_fvc` and `stl_decompose` fail to process anomalous regions common in remote sensing imagery (e.g. No-data, infinite values, cloud cover), which severely compromises downstream analytical accuracy. **3) Inappropriate numerical processing:** Several tools apply arbitrary numerical operations that corrupt the intrinsic physical significance of the data. For example, `calculate_tif_average` and `calc_batch_image_mean_threshold` apply naive min-max linear stretching to physical metrics like NDVI and LST, destroying their absolute physical meanings. In contrast, OpenEarth-Agent overcomes these bottlenecks by leveraging active data perception to dynamically acquire real-time metadata alongside spatio-temporal-modal characteristics. This holistic data comprehension enables the agent to adaptively synthesize highly versatile, robust tools tailored to the specific data context, ensuring robust generalization unconstrained by the limitations of restricted data scenarios.

5.3 Ablation Study

To validate the effectiveness of the multi-agent collaborative architecture and the open knowledge and tool integration modules, we conduct a comprehensive ablation study on the GPT-5-powered OpenEarth-Agent using the OpenEarth-Bench.

Effectiveness of the Multi-Agent Architecture. While a single code-based agent is capable of creating tools and executing tasks through code generation, we conducted ablation studies on other specialized agents to investigate the advantages of multi-agent architecture over a single coding agent. We ablate the Data Summary, Plan & Workflow, and Result Check agents, respectively. The results, reported in Table 4, reveal the indispensable roles of each agent in handling long-horizon tasks within open environments:

Table 4: Ablation study on the multi-agent architecture.

Data Summary	Plan & Workflow	Result Check	Stage-Wise			End-to-End		
			Data Preparation	Feature Extraction	Geospatial Analysis	Data Preparation	Feature Extraction	Geospatial Analysis
✗	✓	✓	84.56	77.85	67.62	81.71	74.16	57.88
✓	✗	✓	83.72	82.21	71.81	81.54	69.13	50.67
✓	✓	✗	78.02	79.86	65.77	75.84	59.73	34.73
✓	✓	✓	**85.40**	**85.27**	**76.66**	**82.38**	**74.83**	**58.72**

1) Data Summary Agent: In the Stage-Wise mode, removing this module leads to a substantial accuracy drop in the feature extraction and geospatial analysis stages. This decline occurs because the agent encounters unknown input data in these mid-to-late stages; lacking a summary module deprives the system of its capability to perceive data characteristics. Notably, its absence has a relatively minor impact during End-to-End evaluations. This demonstrates that in full-pipeline tasks, metadata acquired during the data preparation phase can flow efficiently within the agent system.

2) Plan & Workflow Agent: The absence of this module triggers severe performance degradation, particularly in the End-to-End long-horizon evaluation, where the accuracy of the terminal geospatial analysis stage plummets to 50.67% (an approximate 8% decrease). This strongly indicates that in open environments, agents lacking adaptive planning are highly susceptible to severe "cascading errors" when confronting highly complex EO tasks.

3) Result Check Agent: Removing this module forces the OpenEarth-Agent into a one-shot execution paradigm. Experiments show that without environmental feedback and trial-and-error opportunities, the system experiences a drastic accuracy drop across all stages. This underscores that real-time self-correction mechanisms are essential prerequisites for ensuring high task success rates in open remote sensing environments.

Table 5: Ablation study on open knowledge and tool integration.

Knowledge Integration	Tool Integration	Stage-Wise			End-to-End		
		Data Preparation	Feature Extraction	Geospatial Analysis	Data Preparation	Feature Extraction	Geospatial Analysis
✗	✗	79.69	81.21	72.48	78.02	66.78	47.65
✗	✓	80.87	84.40	73.82	78.86	69.46	51.34
✓	✗	84.56	82.04	74.16	81.20	70.80	52.68
✓	✓	**85.40**	**85.27**	**76.66**	**82.38**	**74.83**	**58.72**

Effectiveness of Knowledge and Tool Integration. To isolate and quantify the performance gains yielded by the external knowledge and tool bases, we perform corresponding ablation experiments. The results in Table 5 demonstrate their crucial and complementary roles:

1) Knowledge Integration: This integration yields the most significant improvements in the data preparation and geospatial analysis stages. During the data acquisition phase, cross-domain knowledge guides the agent to accurately match the required multi-source heterogeneous data. During the analysis phase, domain knowledge provides the agent with indispensable geoscience priors, ensuring the scientific validity of the analytical logic.

2) Multi-Stage Tool Integration: Tool integration predominantly enhances the feature extraction and geospatial analysis stages. When facing feature extraction and complex analytical demands in open environments, the extensive tool repository offers abundant algorithmic references, safeguarding the correctness and robustness of the tools generated by OpenEarth-Agent.

3) Synergistic Effect: When both modules are enabled, the model achieves optimal performance across all metrics, indicating that the integration of knowledge and tools exerts a complementary effect, jointly elevating the agent's overall capabilities.

6 Discussion

6.1 Limitations and Expectations

While OpenEarth-Agent demonstrates strong generalization capabilities in open environments, its dynamic tool creation paradigm fundamentally depends on the underlying LLM's coding and reasoning proficiency. Although code-probing and iterative feedback mechanisms mitigate many errors, the agent may occasionally synthesize tools that are syntactically correct but functionally suboptimal for highly complex geospatial modeling. Furthermore, the continuous cycle of real-time data perception, dynamic DAG planning, and iterative tool refinement requires multiple LLM inference calls. This introduces higher computational overhead and processing latency compared to traditional, static predefined tool-calling pipelines, which may currently limit its deployment in highly time-critical emergency response scenarios.

Moving forward, our future work will focus on two primary expectations. First, we aim to optimize inference efficiency by implementing a robust tool-caching mechanism, allowing successfully synthesized and verified tools to be archived and reused as templates to reduce redundant LLM API calls. Second, we plan to directly integrate physical laws and geospatial topological constraints into the tool synthesis prompts, which will strictly bound the agent's action space and further reduce logical hallucinations in complex Earth Observation tasks.

6.2 Societal Implications and Environmental Considerations

The deployment and operation of OpenEarth-Agent, particularly its reliance on continuous LLM inference for dynamic tool creation and iterative feedback, necessitate substantial computational resources. This intensive computational pipeline inherently leads to significant energy consumption, constituting a critical environmental concern that contributes materially to the carbon footprint and associated ecological impacts. While the framework democratizes complex Earth Observation analysis and aids in monitoring environmental changes, we must critically recognize these externalities.

Adopting sustainable energy solutions can demonstrably reduce the ecological ramifications of autonomous geospatial agents, aligning technological advancement with environmental stewardship. It is incumbent upon the research community to proactively evaluate and address the environmental costs inherent in deploying computationally demanding agentic methodologies. Key mitigation approaches must encompass not only sustainable energy sourcing but also the continuous pursuit of algorithmic optimization—such as implementing efficient tool-caching to minimize redundant generation—and the utilization of energy-efficient, distilled models to reduce overall computational overhead.

7 Conclusion

In this paper, we introduce OpenEarth-Agent, the first tool-creation agent framework tailored for open-environment EO. By shifting the paradigm toward adaptive workflow planning and tool creation conditioned on data and task contexts, and coupling this with open-ended multi-stage tool and cross-domain knowledge integration, OpenEarth-Agent successfully executes end-to-end EO pipelines across multiple domains. Furthermore, to provide a rigorous evaluation platform, we construct OpenEarth-Bench, the first open-environment benchmark for remote sensing agents, comprising 596 full-pipeline application cases across 7 application domains. Extensive evaluations on OpenEarth-Bench, alongside cross-benchmark experiments on Earth-Bench, firmly validate the efficacy of our proposed framework. Notably, it achieves competitive performance using only a minimal set of essential tools compared to baselines relying on full tool calling, and outperforms existing agents when provided with the complete toolset. Ultimately, OpenEarth-Agent establishes a robust and generalizable foundation for the future of autonomous EO in open environments.

References

Xiao An, Jiaxing Sun, Zihan Gui, and Wei He. Choice: benchmarking the remote sensing capabilities of large vision-language models. *arXiv preprint arXiv:2411.18145*, 2024.

Christopher F Brown, Steven P Brumby, Brookie Guzder-Williams, Tanya Birch, Samantha Brooks Hyde, Joseph Mazzariello, Wanda Czerwinski, Valerie J Pasquarella, Robert Haertel, Simon Ilyushchenko, et al. Dynamic world, near real-time global 10 m land use land cover mapping. *Scientific data*, 9(1):251, 2022.

Zhengchao Chen, Haoran Wang, Jing Yao, Pedram Ghamisi, Jun Zhou, Peter M Atkinson, and Bing Zhang. Cangling-knowflow: A unified knowledge-and-flow-fused agent for comprehensive remote sensing applications. *arXiv preprint arXiv:2512.15231*, 2025.

Gheorghe Comanici, Eric Bieber, Mike Schaekermann, Ice Pasupat, Noveen Sachdeva, Inderjit Dhillon, Marcel Blistein, Ori Ram, Dan Zhang, Evan Rosen, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities. *arXiv preprint arXiv:2507.06261*, 2025.

Pengyu Dai, Weihao Xuan, Junjue Wang, Hongruixuan Chen, Jian Song, Yafei Ou, and Naoto Yokoya. Experience-driven multi-agent systems are training-free context-aware earth observers. *arXiv preprint arXiv:2602.02559*, 2026.

Muhammad Danish, Muhammad Akhtar Munir, Syed Roshaan Ali Shah, Kartik Kuckreja, Fahad Shahbaz Khan, Paolo Fraccaro, Alexandre Lacoste, and Salman Khan. Geobench-vlm: Benchmarking vision-language models for geospatial tasks. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7132–7142, 2025.

Minh Kha Do, Kang Han, Phu Lai, Khoa T Phan, and Wei Xiang. Robsense: A robust multi-modal foundation model for remote sensing with static, temporal, and incomplete data adaptability. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 7427–7436, 2025.

Peilin Feng, Zhutao Lv, Junyan Ye, Xiaolei Wang, Xinjie Huo, Jinhua Yu, Wanghan Xu, Wenlong Zhang, Lei Bai, Conghui He, et al. Earth-agent: Unlocking the full landscape of earth observation with agents. *arXiv preprint arXiv:2509.23141*, 2025.

Noel Gorelick, Matt Hancher, Mike Dixon, Simon Ilyushchenko, David Thau, and Rebecca Moore. Google earth engine: Planetary-scale geospatial analysis for everyone. *Remote sensing of Environment*, 202:18–27, 2017.

Haonan Guo, Xin Su, Chen Wu, Bo Du, Liangpei Zhang, and Deren Li. Remote sensing chatgpt: Solving remote sensing tasks with chatgpt and visual models. In *IGARSS 2024-2024 IEEE International Geoscience and Remote Sensing Symposium*, pages 11474–11478. IEEE, 2024.

Huiyang Hu, Peijin Wang, Yingchao Feng, Kaiwen Wei, Wenxin Yin, Wenhui Diao, Mengyu Wang, Hanbo Bi, Kaiyue Kang, Tong Ling, et al. Ringmo-agent: A unified remote sensing foundation model for multi-platform and multi-modal reasoning. *arXiv preprint arXiv:2507.20776*, 2025.

Kaiyu Li, Jiayu Wang, Zhi Wang, Hui Qiao, Weizhan Zhang, Deyu Meng, and Xiangyong Cao. Designing domain-specific agents via hierarchical task abstraction mechanism. *arXiv preprint arXiv:2511.17198*, 2025.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024a.

Fan Liu, Delong Chen, Zhangqingyun Guan, Xiaocong Zhou, Jiale Zhu, Qiaolin Ye, Liyong Fu, and Jun Zhou. Remoteclip: A vision language foundation model for remote sensing. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–16, 2024b.

Akashah Shabbir, Muhammad Akhtar Munir, Akshay Dudhane, Muhammad Umer Sheikh, Muhammad Haris Khan, Paolo Fraccaro, Juan Bernabe Moreno, Fahad Shahbaz Khan, and Salman Khan. Thinkgeo: Evaluating tool-augmented agents for remote sensing tasks. *arXiv preprint arXiv:2505.23752*, 2025.

Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.

Fengxiang Wang, Hongzhen Wang, Zonghao Guo, Di Wang, Yulin Wang, Mingshuo Chen, Qiang Ma, Long Lan, Wenjing Yang, Jing Zhang, et al. Xlrs-bench: Could your multimodal llms understand extremely large ultra-high-resolution remote sensing imagery? In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 14325–14336, 2025.

Zhitong Xiong, Yi Wang, Fahong Zhang, Adam J Stewart, Joëlle Hanna, Damian Borth, Ioannis Papoutsis, Bertrand Le Saux, Gustau Camps-Valls, and Xiao Xiang Zhu. Neural plasticity-inspired multimodal foundation model for earth observation. *arXiv preprint arXiv:2403.15356*, 2024a.

Zhitong Xiong, Fahong Zhang, Yi Wang, Yilei Shi, and Xiao Xiang Zhu. Earthnets: Empowering artificial intelligence for earth observation. *IEEE Geoscience and Remote Sensing Magazine*, 2024b.

Zhitong Xiong, Yi Wang, Weikang Yu, Adam J Stewart, Jie Zhao, Nils Lehmann, Thomas Dujardin, Zhenghang Yuan, Pedram Ghamisi, and Xiao Xiang Zhu. Dofa-clip: Multimodal vision-language foundation models for earth observation. *arXiv preprint arXiv:2503.06312*, 2025.

Wenjia Xu, Zijian Yu, Boyang Mu, Zhiwei Wei, Yuanben Zhang, Guangzuo Li, Jiuniu Wang, and Mugen Peng. Rs-agent: Automating remote sensing tasks through intelligent agent. *arXiv preprint arXiv:2406.07089*, 2024.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Qiangqiang Yuan, Huanfeng Shen, Tongwen Li, Zhiwei Li, Shuwen Li, Yun Jiang, Hongzhang Xu, Weiwei Tan, Qianqian Yang, Jiwen Wang, et al. Deep learning in environmental remote sensing: Achievements and challenges. *Remote sensing of Environment*, 241:111716, 2020.

Zilun Zhang, Tiancheng Zhao, Yulong Guo, and Jianwei Yin. Rs5m and georsclip: A large-scale vision-language dataset and a large vision-language model for remote sensing. *IEEE Transactions on Geoscience and Remote Sensing*, 62:1–23, 2024.